OFFICIAL ENGINEERING INTERFACE CONTRACT

DS3D-HVAC Simulator — LabVIEW Integration Guide

UDP Telemetry Specification, JSON Formats, Error 56 Mitigation, and Master Excel Mappings

PROTOCOL	COMMANDS PORT (INBOUND)	TELEMETRY PORT (OUTBOUND)	TARGET SIMULATOR
UDP / JSON & CSV	**Port 9000 → Unity**	**Port 5005 → LabVIEW**	**DS3D-HVAC Phase 1**

1. System Architecture & Network Topology

The DS3D-HVAC simulation platform operates as a distributed system pairing the **LabVIEW Simulation / Instructor Station** with the **Unity 3D Engine (DS3D)** via lightweight, non-blocking UDP network packets.

Channel	Direction	Port & Host	Format	Description
Commands & Triggers	LabVIEW → Unity	`9000` (127.0.0.1)	`JSON` / Text	Fault trigger scenarios (101–114 active, 115–116 reserved), workflow resets, step skipping, and instructor overrides.
Continuous Sliders	LabVIEW → Unity	`9000` (127.0.0.1)	`JSON` / CSV	16 continuous physical parameters: airflow velocity, condenser/evaporator fan speeds, damper blendshapes.
Scoring Telemetry	Unity → LabVIEW	`5005` (127.0.0.1)	`JSON` / Integer	Event-driven step completions, trainee wrong tool clicks (mistakes counter), and fault completion reports.

Adaptive Remote Endpoint Synchronization

Unity's `UDPManager` has `autoUpdateRemoteIPFromSender = true` enabled. When LabVIEW transmits any command to Port 9000, Unity automatically locks onto LabVIEW's IP address and returns all telemetry to Port 5005 on that exact sender IP.

Architecture Clarification: Direct UDP Sockets vs. Excel File Flow

Runtime Communication: Real-time control and telemetry between LabVIEW and Unity are executed directly over **UDP network sockets** (`9000` / `5005`) with sub-millisecond latency.

Excel's Role: The Master Excel workbook is the **Interface Control Document (ICD) & Test Reporting Template**. Live polling of an `.xlsx` file during runtime is avoided in production because Windows applies exclusive write locks (`SharingViolationException`) when a file is open in Excel/LabVIEW, and disk decompression creates unwanted lag. LabVIEW sends live commands directly to Unity over UDP, and logs final results to Excel.

2. Inbound Commands (LabVIEW → Unity, Port 9000)

LabVIEW transmits commands to Port 9000. Payloads are formatted as lightweight JSON.

2.1 Trigger Procedural Fault Scenario

Instructs Unity to launch a 3D fault scenario. When received, Unity:

- Smoothly slides open both the **Tool Panel** (left) and **Fault Menu** (right).
- Auto-scrolls the horizontal fault menu directly to the selected fault tab.

- Focuses the 3D camera onto the target equipment.
- Auto-scrolls the tool rack to the required tool for Step 1 and highlights it.
- Dispatches an immediate `fault_started` telemetry packet back to LabVIEW.

```
// 1. Standard Explicit JSON Command (Recommended)
{
  "command": "trigger_fault",
  "fault_code": 101
}

// 2. Shorthand JSON Format
{
  "fault": 101
}
```

2.2 Reset Simulator (JSON Prompt Only)

Aborts any active fault workflow, returns 3D tools to storage, slides UI panels closed, and restores default camera view. *Note: The Reset button has been removed from the trainee UI; reset is triggered remotely via this JSON command.*

```
{
  "command": "reset"
}
```

2.3 Compression Highlights & Idle Arrow Visibility Control

In idle state, HVAC refrigerant circulation arrows and compression highlights (`CompresserHighLight`) are active. When any fault triggers, they turn off automatically. LabVIEW can also explicitly toggle them on or off at any time:

```
// 1. Explicit JSON Command Format
{
  "command": "set_compression_highlights",
  "state": true  // true = Show Highlights, false = Hide Highlights
}

// 2. Direct Shorthand Property Format
{
  "compression_highlights": true
}

// 3. Arrow Visibility Control Command
{
  "command": "set_arrows",
  "state": true  // true = Show Arrows, false = Hide Arrows
}
```

> **Trainee UI Simplification & Procedural Integrity**
>
> To enforce strict procedural training, the **Skip**, **Back Step**, and **Reset** buttons have been removed from the trainee interface. The **Submit** button is centered at the bottom of the remedial steps panel. Remote step-skipping and back-stepping have also been disabled so trainees must complete each procedural step in sequence.

2.4 Master Composite JSON Schema (Speed, Amplitude, Boolean, Fault, Spare)

LabVIEW transmits comprehensive simulation parameters, individual compressor highlights, fault trigger/reset actions, and spare telemetry registers via a single unified payload over Port 9000:

```
{
  "Hvac": {
    "Speed": [
      { "Name": "BlueArrowOut",   "Value": 5 },
      { "Name": "Fan1",           "Value": 0 },
      { "Name": "Fan2",           "Value": 0 },
      { "Name": "GreenArrowIN1",  "Value": 5 },
      { "Name": "GreenArrowIN2",  "Value": 5 },
      { "Name": "YellowArrowIn",  "Value": 5 }
    ],
    "Amplitude": [
      { "Name": "RedArrow_Inlet",    "Value": 0.0 },
      { "Name": "RedArrow_Outlet",   "Value": 0.0 },
      { "Name": "BlueArrow_Outlet",  "Value": 0.0 },
      { "Name": "GreenArrow_Inlet",  "Value": 0.0 },
      { "Name": "YellowArrow_Inlet", "Value": 0.0 },
      { "Name": "FADL",              "Value": 0.0 },
      { "Name": "FADR",              "Value": 0.0 },
      { "Name": "RAD",               "Value": 0.0 }
    ],
    "Boolean": [
      { "Name": "DS3D_OFF",              "Value": 0 },
      { "Name": "ShowHideObjects",       "Value": false },
      { "Name": "Compresor 1_Highlights", "Value": true },
      { "Name": "Compresor 2_Highlights", "Value": true },
      { "Name": "Compresor 3_Highlights", "Value": true },
      { "Name": "Compresor 4_Highlights", "Value": true }
    ],
    "Fault": [
      { "Name": "Fault_Code_Trigger",    "Value": 0 },
      { "Name": "Fault_Cause_Trigger",   "Value": 0 },
      { "Name": "Fault_Command_Action",  "Value": 0 }
    ],
    "Spare": [
      { "Name": "Spare_24", "Value": 0 },
      { "Name": "Spare_25", "Value": 0 },
      { "Name": "Spare_26", "Value": 0 },
      { "Name": "Spare_27", "Value": 0 },
      { "Name": "Spare_28", "Value": 0 },
      { "Name": "Spare_29", "Value": 0 },
      { "Name": "Spare_30", "Value": 0 }
    ]
  }
}
```

Field Functionality & Behavior:

- **Fault Array:**
  - `Fault_Code_Trigger`: Target Fault ID (e.g. `101`, `102`).
  - `Fault_Cause_Trigger`: Target Sub-cause identifier (e.g. `1`).
  - `Fault_Command_Action`: Action strobe control:

        - `0` = **Idle / No Action** (normal continuous streaming).
        - `1` = **Trigger Fault** (starts scenario for `Fault_Code_Trigger`, slides panels open, focuses camera, turns off idle highlights/arrows).
        - `2` = **Reset Fault** (aborts active scenario, closes UI panels, restores camera, turns highlights/arrows back on).
- **Boolean Array:**
    - `DS3D_OFF`: `0` / `false` = normal run; `1` / `true` = close application.
    - `ShowHideObjects`: `false` / `true` toggles equipment casing visibility.
    - `Compresor 1..4_Highlights`: Independent boolean toggles for `Compression1`, `Compression2`, `Compression3`, and `Compression4` 3D highlights. Mixed integer (`0/1`) and boolean (`true/false`) representations are both fully supported.
- **Spare Array:** Reserved expansion channels (`Spare_24` through `Spare_30`) safely parsed and maintained in Unity memory.

3. Outbound Telemetry (Unity → LabVIEW, Port 5005)

Unity transmits real-time execution feedback and training scoring over UDP port 5005 as UTF-8 encoded JSON objects.

3.1 Event: `fault_started`

Dispatched immediately when a scenario starts. Confirms total steps, turns off idle highlights/arrows, and initializes LabVIEW counters.

```
{
  "event": "fault_started",
  "fault_code": 101,
  "total_steps": 14,
  "status": "started",
  "compression_highlights": false,
  "arrows_active": false
}
```

3.2 Event: `step_completed`

Dispatched whenever a step is completed by the trainee OR auto-demonstrated after 3 mistakes. LabVIEW reads this to track progress and score mistakes.

```
{
  "event": "step_completed",
  "fault_code": 101,
  "step_number": 1,
  "wrong_attempts": 2,
  "total_steps": 14,
  "status": "success",
  "is_fault_completed": false,
  "compression_highlights": false,
  "arrows_active": false
}
```

JSON Key	LabVIEW Type	Field Semantics & Scoring Logic
`event`	String	Always `"step_completed"`. Used to route in LabVIEW Case Structure.
`fault_code`	Numeric (I32)	Active fault scenario identifier (e.g. `101`).
`step_number`	Numeric (I32)	1-based index of the step that was just accomplished (`1, 2, 3...`).

JSON Key	LabVIEW Type	Field Semantics & Scoring Logic
`wrong_attempts`	Numeric (I32)	**Trainee mistakes:** Count of wrong tool clicks prior to success. If `wrong_attempts >= 3`, indicates the 3-mistake limit triggered auto-demonstration.
`total_steps`	Numeric (I32)	Total steps required for this procedure.
`is_fault_completed`	Boolean	`true` if this was the final step; `false` if additional steps remain.
`compression_highlights`	Boolean	`true` if highlights are visible; `false` if deactivated during fault.
`arrows_active`	Boolean	`true` if arrows are visible; `false` if deactivated during fault.

3.3 Event: `fault_completed`

Dispatched upon completing the final step of a procedure. Restores idle compression highlights and arrows, and contains full mistake audit.

```
{
  "event": "fault_completed",
  "fault_code": 101,
  "status": "success",
  "total_steps": 14,
  "wrong_attempts_per_step": [0, 2, 1, 0, 3, 0, 0, 1, 0, 0, 0, 0, 0, 0],
  "is_fault_completed": true,
  "compression_highlights": true,
  "arrows_active": true
}
```

3.4 Events: `compression_highlights_changed` & `arrows_changed`

Dispatched whenever an instructor command toggles highlights or arrows explicitly:

```
{
  "event": "compression_highlights_changed",
  "compression_highlights": true,
  "status": "enabled"
}

{
  "event": "arrows_changed",
  "arrows_active": true,
  "status": "enabled"
}
```

3.5 Event: `fault_reset`

Dispatched when the instructor sends a reset command, confirming return to idle state:

```
{
  "event": "fault_reset",
  "status": "idle",
  "compression_highlights": true,
  "arrows_active": true
}
```

4. LabVIEW Error 56 Mitigation (UDP Read Timeout)

> **Understanding LabVIEW Error 56**
>
> LabVIEW **Error 56** is a standard socket timeout. It occurs when `UDP Read.vi` waits for incoming packets longer than its configured timeout (default: 25,000 ms). Because fault telemetry is *event-driven* (packets are only sent when a user clicks a tool or finishes a step), timeouts are normal while the trainee inspects the 3D model.

Recommended LabVIEW Architecture:

1. **Set Timeout Low:** Configure `UDP Read.vi` timeout to **100 ms – 200 ms**.
2. **Case Structure Error Filter:** Wire the error cluster from `UDP Read.vi` to a Case Structure:
   - **Error Case:** Use `Unbundle by Name` to check `code == 56`.
     - If `code == 56`: Call `Clear Error.vi`. *Do not halt execution.*
     - If `code != 56`: Propagate genuine network faults (e.g. socket closure).
   - **No Error Case:** Pass the incoming byte array into `JSON to Cluster.vi`.
3. **Sample-and-Hold Shift Register:** Store the previous valid telemetry cluster in a While Loop shift register. When Error 56 occurs, output the retained values so UI indicators never flicker or reset to zero.

5. Fault Catalog Reference (Scenarios 101 – 114 Active + 115 – 116 Reserved)

> **Architecture Clarification: 14 Active 3D Workflows in Unity vs. 16 Slots in Master Excel**
>
> **Why Unity UI shows 14 faults (101–114):** The Unity DS3D simulator contains exactly **14 fully authored interactive 3D fault scenarios** (Faults 101 through 114). The Unity interface dynamically builds its tab list directly from `FaultManager.faults.FaultList` (14 items), ensuring only scenarios with complete 3D models, tool targets, animations, and camera tracks appear to the trainee.
>
> **Why the Master Excel & Documentation mention 116:** In the railway specification team's original master Excel workbook (sheet `fault List` and sheet `DATA TRANSFER` rows 828–843), 16 slot addresses (101–116) were allocated. **Faults 115 and 116 are Reserved / Spare Placeholders** reserved for future subsystem additions. They have no 3D animation assets authored in Unity at this stage.

Fault workflows implemented in Unity DS3D alongside reserved Excel allocation slots:

Code	Subsystem	Fault Title / Scope	Steps	JSON Command Payload
101	Refrigeration Circuit	No HVAC operations	14	`{"command":"trigger_fault","fault_code":101}`
102	Blower & Ventilation	No ventilation	22	`{"command":"trigger_fault","fault_code":102}`
103	Ventilation Circuit	Reduced ventilation	15	`{"command":"trigger_fault","fault_code":103}`
104	Compressor / Pressure	Suction low & discharge low (Circuit 1)	15	`{"command":"trigger_fault","fault_code":104}`
105	Expansion / Pressure	Suction low & discharge low (Circuit 2)	14	`{"command":"trigger_fault","fault_code":105}`
106	Differential Pressure	High suction and high discharge	26	`{"command":"trigger_fault","fault_code":106}`

Code	Subsystem	Fault Title / Scope	Steps	JSON Command Payload
107	Compressor Unit	Compressor not operating	40	`{"command":"trigger_fault","fault_code":107}`
108	Cooling Capacity	Reduced cooling	33	`{"command":"trigger_fault","fault_code":108}`
109	Heating Subsystem	No Heating	15	`{"command":"trigger_fault","fault_code":109}`
110	Thermal Regulation	Over Cooling	8	`{"command":"trigger_fault","fault_code":110}`
111	Thermal Regulation	Over Heating	7	`{"command":"trigger_fault","fault_code":111}`
112	Emergency Systems	No emergency ventilation	14	`{"command":"trigger_fault","fault_code":112}`
113	Safety & Sensors	Smoke identified in saloon	13	`{"command":"trigger_fault","fault_code":113}`
114	Mechanical Diagnostics	Compressor noisy | Condenser Fan / Blower	36	`{"command":"trigger_fault","fault_code":114}`
115	Auxiliary / Expansion	*[Reserved / Spare in Master Excel]*	—	*Reserved Excel slot (No 3D asset authored in Unity)*
116	Auxiliary / Expansion	*[Reserved / Spare in Master Excel]*	—	*Reserved Excel slot (No 3D asset authored in Unity)*

6. Master Excel Workbook Cell Mappings

Workbook: **`J886_Software_HVAC_phase1-V1_Fault_Management_Updated.xlsx`**

6.1 Sheet `DATA TRANSFER` — Inbound Telemetry (Unity → LabVIEW, Rows 821–854)

Row	Variable Name	Data Type	Cell Ref	Description
821	`Active_Fault_Code`	Integer	`F821`	Currently active scenario code (101–114 active, 115–116 reserved).
822	`Current_Step_Number`	Integer	`F822`	1-based index of the currently active/accomplished step (0 = unstarted/reset).
823	`Total_Steps`	Integer	`F823`	Total steps required for active scenario.
824	`Wrong_Attempts`	Integer	`F824`	Mistakes counter: cumulative wrong tool clicks on this step.
825	`Step_Accomplished`	Boolean	`F825`	Status pulse: `1` when step finishes successfully.
826	`Fault_Completed`	Boolean	`F826`	Status flag: `1` when full fault procedure is finished.
827–840	`Fault_101_Status` … `114`	Boolean	`F827:F840`	Active scenario flags for Unity 3D Faults 101 through 114 (14 continuous rows).
841–842	`Fault_115_Status` & `116`	Boolean	`F841:F842`	*Reserved / Spare slots* allocated in master Excel for future expansion.
843	`Compression_Highlights_Active`	Boolean	`F843`	Compression highlights visibility (`0` = Inactive during fault, `1` = Active in idle).
844	`Arrows_Active`	Boolean	`F844`	Refrigerant arrow circulation visibility (`0` = Inactive during fault, `1` = Active in idle).
845–853	Spare (9 rows)	Boolean	`F845:F853`	Spare Boolean slots grouped contiguously with active Booleans.
854	`Telemetry_Event`	String	`F854`	Last event: `"fault_started"`, `"step_completed"`, `"fault_completed"`, `"fault_reset"`.

6.2 Sheet `DATA TRANSFER` — Outbound Commands (LabVIEW → Unity, Rows 835–840)

Row	Variable Name	Data Type	Formula / Link	Description
835	`DS3D_OFF`	Integer	Direct Value	Application shutdown signal (`0`=Running, `1`=Quit).
836	`Fault_Code_Trigger`	Integer	`='DATA TRANSFER'!F1209`	Dispatches selected Fault Code (101–114 active) to Unity.

Row	Variable Name	Data Type	Formula / Link	Description
837	`Fault_Cause_Trigger`	Integer	`='DATA TRANSFER'!F1210`	Dispatches Cause Number (10101–11401 active) to Unity.
838	`Fault_Command_Action`	Integer	`='DATA TRANSFER'!F1211`	Action: `0`=Idle, `1`=Trigger Fault, `2`=Reset Fault.
839	`ShowHideObjects`	Boolean	Direct Value	Toggle component visibility in 3D scene (`0`=Hide, `1`=Show).
840	`Compression_Highlights`	Boolean	`='DATA TRANSFER'!F1212`	Explicitly toggle compression highlights (`0`=Hide, `1`=Show).

6.3 Sheet `DATA TRANSFER` — Instructor Control Cells (Rows 1209–1212)

Row	Variable Name	Data Type	Cell Ref	Instructor Station UI Function
1209	`Fault_Number`	Integer	`F1209`	Instructor sets desired fault scenario (101–114 active).
1210	`Cause_Number`	Integer	`F1210`	Instructor sets specific cause code (10101–11401 active).
1211	`Fault_Command_Action`	Integer	`F1211`	Command: `1` to Trigger Fault, `2` to Reset Fault.
1212	`Compression_Highlights`	Boolean	`F1212`	Toggle Compression Highlights Visibility (`0`=Hide, `1`=Show).

7. Verification & Testing Utilities

To verify the entire integration without opening LabVIEW, use the bundled Python test bridge:

- **Double-Click Launcher:** `Run_Fault_Test_Bridge.bat` in the `DS3D-HVAC` project root.
- **CLI Execution:** `python hvac_fault_test_bridge.py`
- **Features:** Binds to Port 5005, dispatches commands to Port 9000, displays live color-coded telemetry of steps and mistakes, and writes live values directly back into the master Excel file.